TP1 IA COMPTE RENDU COULIBALY Sidiné

1. Fonction d’évaluation

Pour choisir un coup, l’IA doit pouvoir déterminer si une position est favorable ou non. J’ai donc utilisé une fonction d’évaluation basée sur deux critères : le nombre de pièces restantes et leur progression sur le plateau.

La différence de nombre de pièces est multipliée par 10, tandis que la différence de progression est multipliée par 5. Un score positif indique une position favorable au joueur évalué, alors qu’un score négatif indique une position défavorable.

La progression est calculée à partir de la pièce la plus avancée dans chaque colonne. Lorsqu’un camp ne possède aucune pièce dans une colonne, une distance maximale lui est attribuée pour cette colonne.

J’ai testé la fonction sur plusieurs positions au cours d’une partie. Les scores obtenus correspondaient aux valeurs attendues, et le score de X était toujours l’opposé de celui de O. Ces vérifications ont permis de confirmer le bon fonctionnement de l’évaluation dans les situations testées.

Le programme attribue également des valeurs particulières aux victoires et aux défaites. Cela permet à l’IA de privilégier les positions gagnantes plutôt que de se baser uniquement sur le nombre de pièces ou leur progression.

2. Résultats obtenus avec Minimax

J’ai commencé par tester l’algorithme Minimax sans coupure de branches. Les résultats obtenus sont présentés dans le tableau suivant.

Profondeur	Résultat	Nœuds moyens par coup	Temps moyen par coup
1	Victoire de X au 97e tour	45,91	0,000084 s
2	Match nul après 400 tours	4 731,19	0,007689 s
3	Victoire de X au 65e tour	225 956,02	0,356178 s

On remarque que le nombre de nœuds explorés augmente très rapidement avec la profondeur. À la profondeur 1, l’IA examine environ 46 nœuds par coup. À la profondeur 3, elle en examine près de 226 000.

Le temps de calcul augmente également. Il passe d’une durée presque imperceptible à environ 0,36 seconde par coup à la profondeur 3.

Cette différence s’explique par le nombre de déplacements possibles à chaque étape. Plus la profondeur augmente, plus l’IA doit examiner de positions pour choisir son coup. La recherche devient donc rapidement coûteuse en temps de calcul.

3. Résultats obtenus avec Alpha-Bêta

J’ai ensuite ajouté les coupures Alpha-Bêta afin d’éviter d’explorer les branches qui ne peuvent plus modifier la décision. Les tests ont été réalisés avec les mêmes profondeurs.

Profondeur	Résultat	Nœuds moyens par coup	Temps moyen par coup
1	Victoire de X au 97e tour	45,91	0,000088 s
2	Match nul après 400 tours	1 531,97	0,002613 s
3	Victoire de X au 65e tour	20 736,62	0,032987 s

À la profondeur 1, les deux algorithmes explorent le même nombre de nœuds. Le temps mesuré avec Alpha-Bêta est même légèrement supérieur, mais la différence est trop faible pour être significative. À cette profondeur, les possibilités de coupure sont limitées.

À la profondeur 2, le gain devient plus intéressant. Alpha-Bêta explore environ trois fois moins de nœuds que Minimax et réduit le temps moyen de calcul d’environ 66 %.

C’est à la profondeur 3 que la différence est la plus importante. Le nombre moyen de nœuds passe d’environ 226 000 à 20 700, tandis que le temps moyen passe de 0,356 seconde à environ 0,033 seconde par coup.

Ces résultats montrent qu’Alpha-Bêta permet d’explorer le même niveau de profondeur en effectuant beaucoup moins de calculs.

4. Comparaison des performances

Pour mieux comparer les deux algorithmes, j’ai calculé les gains obtenus avec Alpha-Bêta.

Profondeur	Réduction du nombre de nœuds	Réduction du temps de calcul
1	0 %	Pas de gain significatif
2	Environ 68 %	Environ 66 %
3	Environ 91 %	Environ 91 %

À la profondeur 3, Minimax explore environ 10,9 fois plus de nœuds qu’Alpha-Bêta. Le temps de calcul est également environ 10,8 fois plus élevé.

Le gain augmente donc avec la profondeur de recherche. Plus l’arbre de recherche est important, plus les coupures peuvent éviter des calculs inutiles.

Les résultats varient toutefois selon les positions. Dans la comparaison coup par coup à la profondeur 3, le rapport entre les nombres de nœuds explorés varie d’environ 4,6 à 36,6. Cela montre que l’efficacité des coupures dépend de la situation et de l’ordre dans lequel les coups sont examinés.

Les deux méthodes ont obtenu les mêmes résultats dans les parties comparées : victoire de X au 97e tour à la profondeur 1, match nul après 400 tours à la profondeur 2 et victoire de X au 65e tour à la profondeur 3. Les journaux disponibles montrent également que la partie à profondeur 3 s’est déroulée coup par coup de la même manière avec les deux algorithmes.

5. Observation des parties

Les parties permettent aussi de mieux comprendre les limites de l’IA.

À la profondeur 1, l’IA anticipe peu les conséquences de ses déplacements. Plusieurs fois, O avance une pièce au contact de l’adversaire et la perd au coup suivant. L’IA ne prend donc pas toujours en compte les dangers immédiats de ses décisions.

À la profondeur 2, les déplacements sont plus prudents, mais les deux camps finissent par répéter les mêmes coups. La partie observée se termine après 400 tours sans vainqueur.

Un cycle de quatre déplacements apparaît en fin de partie : `J4J5`, `I9J9`, `J5J4` et `J9I9`. Les pièces reviennent régulièrement à leur position précédente et le score reste constant. Ce comportement montre que l’IA peut avoir du mal à sortir d’une situation où plusieurs coups ont une valeur similaire.

À la profondeur 3, l’IA anticipe davantage les conséquences de ses actions. Dans la partie corrigée, X joue `I2J1` au 63e tour, puis `J1J0` au 65e tour, ce qui lui permet d’atteindre la ligne de fond adverse et de gagner.

Ces observations montrent que la profondeur de recherche influence les décisions de l’IA. Cependant, une seule partie par profondeur ne suffit pas pour affirmer que la profondeur est la seule cause des différences observées.

6. Axes d’amélioration

Même si les résultats obtenus avec Alpha-Bêta sont satisfaisants, certains points pourraient être améliorés. Je retiens principalement deux pistes : limiter les répétitions de coups et rendre la recherche plus efficace.

6.1. Améliorer les décisions à profondeur 2

Le cycle observé à la profondeur 2 est l’un des problèmes les plus visibles du programme. L’IA répète les mêmes déplacements jusqu’à atteindre la limite de 400 tours.

Une explication possible vient de la fonction d’évaluation. Comme elle prend principalement en compte la pièce la plus avancée de chaque colonne, certains déplacements ne changent pas le score. L’IA peut donc considérer plusieurs coups comme équivalents et choisir toujours le même.

Pour améliorer ce comportement, il serait intéressant de prendre en compte la progression de toutes les pièces, avec un coefficient faible afin que ce critère ne prenne pas trop d’importance par rapport au matériel et à la progression déjà évalués. Il faudrait alors recalculer les bornes de la fonction d’évaluation.

Une autre solution, plus simple à mettre en place, serait de pénaliser certains retours immédiats vers une position précédente. L’IA pourrait ainsi éviter de répéter un déplacement lorsqu’un autre coup présente une valeur équivalente.

Ces modifications permettraient de limiter les cycles et d’encourager l’IA à choisir des déplacements plus utiles. Il faudrait ensuite refaire plusieurs parties pour vérifier si le nombre de matchs nuls diminue.

6.2. Réduire davantage le temps de calcul

Alpha-Bêta réduit déjà fortement le nombre de nœuds explorés, mais son efficacité dépend de l’ordre dans lequel les coups sont examinés.

Une amélioration possible serait de tester en priorité les coups qui semblent les plus intéressants, par exemple les prises ou les déplacements qui rapprochent les pièces de la ligne adverse. Si les meilleurs coups sont examinés plus tôt, l'algorithme peut effectuer davantage de coupures et éviter d'explorer certaines branches.

Il serait également possible d'améliorer la gestion de la mémoire. Actuellement, le programme alloue et libère de la mémoire pour simuler les déplacements. Réutiliser des plateaux de travail pourrait réduire ce coût, à condition de restaurer correctement le plateau après chaque simulation.

Pour mesurer l'intérêt de ces changements, il faudrait comparer le nombre de nœuds et le temps moyen par coup avant et après chaque modification. Cela permettrait de vérifier si les améliorations apportent un gain réel.

7. Conclusion

Ce projet m'a permis de comparer deux méthodes de recherche utilisées pour les jeux de stratégie et de constater leur différence en pratique.

Minimax est relativement simple à mettre en place, mais son coût de calcul augmente très rapidement avec la profondeur. Alpha-Bêta permet de limiter cette augmentation en évitant d'explorer certaines branches qui ne peuvent plus influencer le choix final.

Les mesures montrent qu'à la profondeur 3, Alpha-Bêta réduit d'environ 91 % le nombre moyen de nœuds explorés et le temps de calcul, tout en conservant le même résultat que Minimax dans la partie observée.

Les tests ont également mis en évidence certaines limites de l'IA, notamment les répétitions de déplacements à la profondeur 2 et les décisions parfois peu efficaces lorsque la recherche est trop courte.

Pour la suite, je privilégierais d'abord la limitation des répétitions, puis l'amélioration de l'ordre d'exploration des coups. Ces deux changements permettraient de travailler à la fois sur la qualité des décisions et sur les performances du programme, sans nécessairement augmenter la profondeur de recherche.